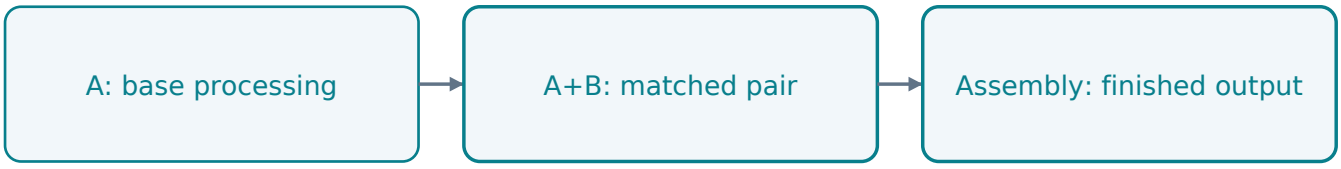
Reading guide

From Production to Intelligent Coordination

Historical PI-on edition. For the latest PI-off strategy, see the M4 algorithm booklets and root README.

Imagine a desk-lamp factory. Line A makes bases, B makes heads, and assembly needs both. If orders rise, which line should receive more WIP? If bases are already piling up, will further releases actually help?



The project starts from these simple-looking but interdependent questions. The algorithm acts as production supervisor, adjusting three WIP caps every 10 minutes to track throughput targets while controlling inventory and adjustment costs.

> The lamp factory is a teaching analogy. The code models an abstract A+B assembly system. Diagrams and action examples are explanatory; experiment charts use saved actual results. Illustrative curves are not presented as measured trajectories.

Suggested reading order

Topic	Pages	What you will understand
Problem and environment	2-5	Why more releases are not always better; how simulation runs
Objective and base algorithms	6-7	Reward calculation and original MADQN learning
M4 architecture improvements	8-12	Shared information, memory, coordination, residuals and ensembles
Experiments and results	13-16	Baselines, convergence, performance and ablation evidence
Contribution scope and review	17-18	Supported conclusions and accurate project descriptions

Estimated reading time: 25-35 minutes. Each chapter starts with production intuition, explains implementation details and highlights common misunderstandings.

Chapter 1 / Problem

The Factory Needs Matched Output

Research subject: dynamic WIP-cap allocation, not individual machine-speed control.

If A produces many parts and B few, assembly still lacks complete sets. A busy branch may simply build inventory. Plant-wide success depends on finished output meeting the target.



Figure 1. Mismatch illustration: three A parts cannot replace one B part. Not an experiment screenshot.

Three obvious actions, each with a cost

Action	Possible outcome	The real question
Increase all caps	More WIP enters, but bottlenecks may prevent proportional throughput growth	Is WIP insufficient, or is capacity insufficient?
Decrease all caps	Inventory falls, but lines may starve and machines wait	Does inventory saving sacrifice tracking accuracy?
React only to current output error	Repeated increases can cause overshoot, followed by repeated decreases	Has the previous adjustment taken effect yet?

The full research question

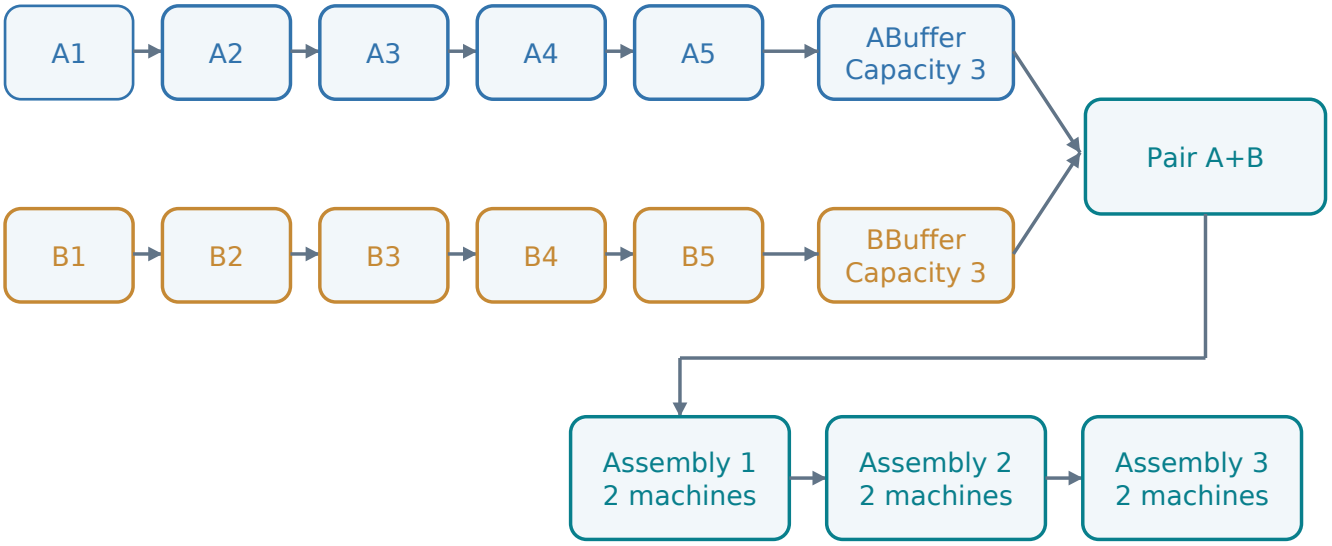
In an A+B synchronized assembly system with finite buffers, stochastic processing and changing targets, use observable state to adapt three local CONWIP caps. Track rolling finished-product throughput while reducing WIP and frequent adjustments.

The controlled quantity is permitted WIP circulating in each loop. The controller does not directly change machine speed, machine count or individual job sequencing.

Chapter 2 / Environment

A Small Factory with Three Loops

Environment: PairV2Env. Discrete-event simulation models processing, queues, pairing and blocking. [S1]



After pairing, process through assembly stages 1 → 2 → 3

Figure 2. Actual process topology. Paired material enters assembly stage 1. Each of three assembly stages has two parallel machines.

Section	Stages and machines	Baseline mean processing time (minutes)
A branch	5 stages, 1 machine each	7, 8, 7, 8, 7
B branch	5 stages, 1 machine each	8, 7, 8, 7, 7
Assembly line	3 stages, 2 machines each	12, 16, 12

Stages have FIFO queues. A/B parts enter their finished-parts buffers after processing. Full buffers can block branch-end machines from releasing parts. Assembly admits a pair only when both parts and assembly-cap allowance are available.

Nominal bottleneck capacity is approximately 7.5 parts/hour under baseline conditions. Actual output also depends on WIP caps, randomness, pairing waits and finite buffers. Nominal capacity is not actual throughput.

Chapter 2 / Control actions

CONWIP: Issue Production Passes

Separate local caps manage A, B and assembly, rather than one plant-wide shared token pool. [S1]

Think of cap as a number of passes. cap_A=3 permits up to three parts to occupy A-loop slots. A part leaving releases a slot, allowing replacement work. A/B buffers have separate capacity limits.

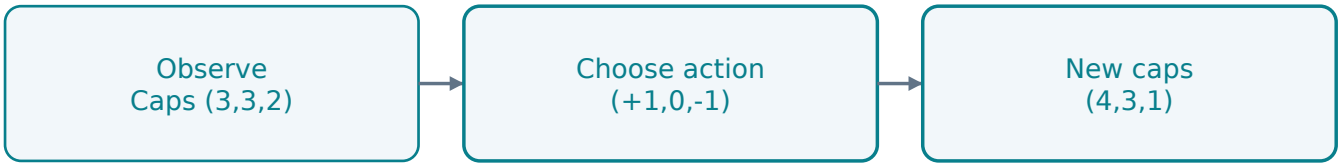


Figure 3. Cap update illustration. Code indices 0/1/2 mean decrease by 1, hold, increase by 1; indices are not increments.

Reducing a cap does not delete parts

If assembly already has 2 sets in process and its cap falls from 2 to 1, both sets continue. New releases pause until WIP falls naturally. Cap constrains releases; instantaneous WIP may temporarily exceed a reduced cap.

Parameter	Current value	Intuitive meaning
Initial caps	(3,3,2)	Provide basic flow through all three units
Cap bounds	0-8 per line	Limit releases or pause replenishment
Decision interval	10 minutes	Let the factory run between decisions
Joint action count	27	Three choices per line: 3×3×3
Control horizon	3600 minutes / 360 steps	Evaluate 60 hours of control per episode
Warm-up	720 minutes	Run 12 hours before evaluating control

With demand_gated=False, releases are mainly cap-constrained, without an additional no-order/no-release gate. Targets affect control through reward; overproduction also incurs tracking cost.

Chapter 3 / Challenges

Why This Is Not Simple Addition and Subtraction

Pairing constraints, delays, randomness and regime changes interact.

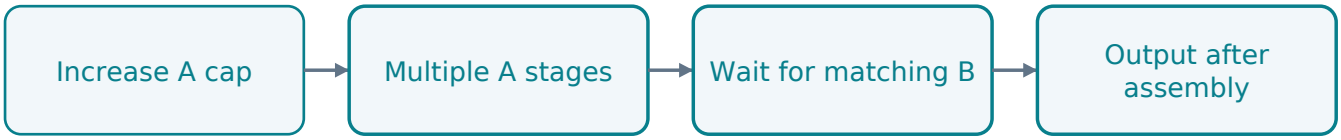


Figure 4. Delay chain. Actions affect releases first, then finished output after processing and pairing. Arrows do not imply fixed durations.

This separates action from reward. A present shortage may reflect insufficient releases several steps ago or simply slow processing. Repeatedly increasing caps before earlier effects arrive can cause overreaction.

Six regimes test the controller

Control time (minutes)	Target (parts/hour)	Change
0-600	3	Baseline processing
600-1200	6	Higher target
1200-1800	5	A/B Gamma shape: 9 → 2
1800-2400	5	Variability restored; B stage 3 mean +15%
2400-3000	5	B restored; A stage 2 mean +15%
3000-3600	3	Lower target; processing means restored

Smaller Gamma shape means more variability at the same mean. A/B coefficients of variation rise from approximately 0.33 to 0.71. Mean speed does not directly slow, but completion timing becomes less predictable and pairing waits may increase.

Change times are predefined. Test seeds vary random processing samples, not the entire regime schedule. Results show adaptation to randomness within this scenario, not effectiveness for every unknown demand curve.

Chapter 4 / Objective

Reward: Calculate the Plant's Total Cost

Tracking, inventory and adjustments determine one shared team reward. [S1]



Each step's reward is the negative sum of these costs. Tracking uses squared relative throughput error. Inventory uses mean part-equivalent WIP over the 10-minute interval divided by 38. Action cost uses the mean absolute cap change across three units.

r = -[(throughput - window target) / max(window target,1)]² - 0.05 × interval mean WIP / 38 - 0.005 × Σ|cap change| / 3. Scheduled changes align with decision boundaries; code also handles within-interval regime splits.

A hand-calculation example (illustrative)

Suppose rolling throughput is 4.5 parts/hour, the window target 5, mean WIP 15, and one line increases its cap by 1. Tracking cost is 0.01, inventory cost approximately 0.01974, adjustment cost approximately 0.00167. Reward is approximately -0.03140, or -31.40 after multiplying by 1000.

This explains the trade-off between smaller error and lower inventory. Weights express operating preferences. These results use the fixed stated weights and do not automatically transfer to a different cost structure.

A two-hour window reduces noise but adds lag

Throughput uses completions over the last 120 minutes, and the reference averages target history over the same window. When the target jumps from 3 to 6, the scoring reference transitions gradually rather than revaluing the entire past window at 6. Observations include both current and window targets.

Metric	Correct interpretation
reward×1000	Mean per-step episode reward ×1000, aggregated across episodes; higher is better
NRMSE	Square root of the episode mean squared tracking error; lower is better
Mean WIP	One part per A/B WIP; two per assembly set; plus buffer parts
Cap-change points	Decisions where any cap changes, not the sum of three lines' adjustment magnitudes

Chapter 5 / Starting point

Original MADQN: Three Learning Supervisors

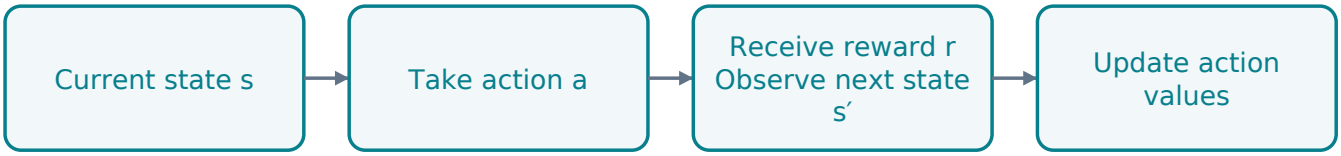
One local Q network per line, sharing team reward but separately estimating action values. [S2]



Figure 5. One-line original-MADQN decision structure. Each of the three lines has such a network.

Q estimates the expected future return from taking an action now. Training gathers experience through repeated simulation. Execution selects the highest-valued legal action. Q is neither predicted throughput nor a success probability.

How does one learning update happen?



The original implementation stores state, action, reward and next state in replay, then samples experiences randomly. Double-DQN-style targets use the online network for next-action selection and the target network for evaluation, reducing some overestimation.

What does it know, and what is missing?

Local agents are not entirely blind to the plant. Their 45-D observations include own WIP/cap, shared buffers, rolling throughput, targets, recent busy/idle/blocked summaries and stage states. However, they lack complete broadcasts of all units' WIP/operating summaries and explicit observation-sequence memory.

During independent learning, teammates' policies also change. Outcomes depend on all three agents, not only one. Shared reward reports whether the team did well but does not identify which decision helped.

This motivates M4: add cross-line and temporal information, strengthen team-value learning, and exploit PI knowledge and independent trained models.

Chapter 6 / M4 overview

Connect Production Information, Experience and Learning

This historical PI-on M4 retains QMIX. Independent Q ensembling does not remove centralized training. [S3, S4]

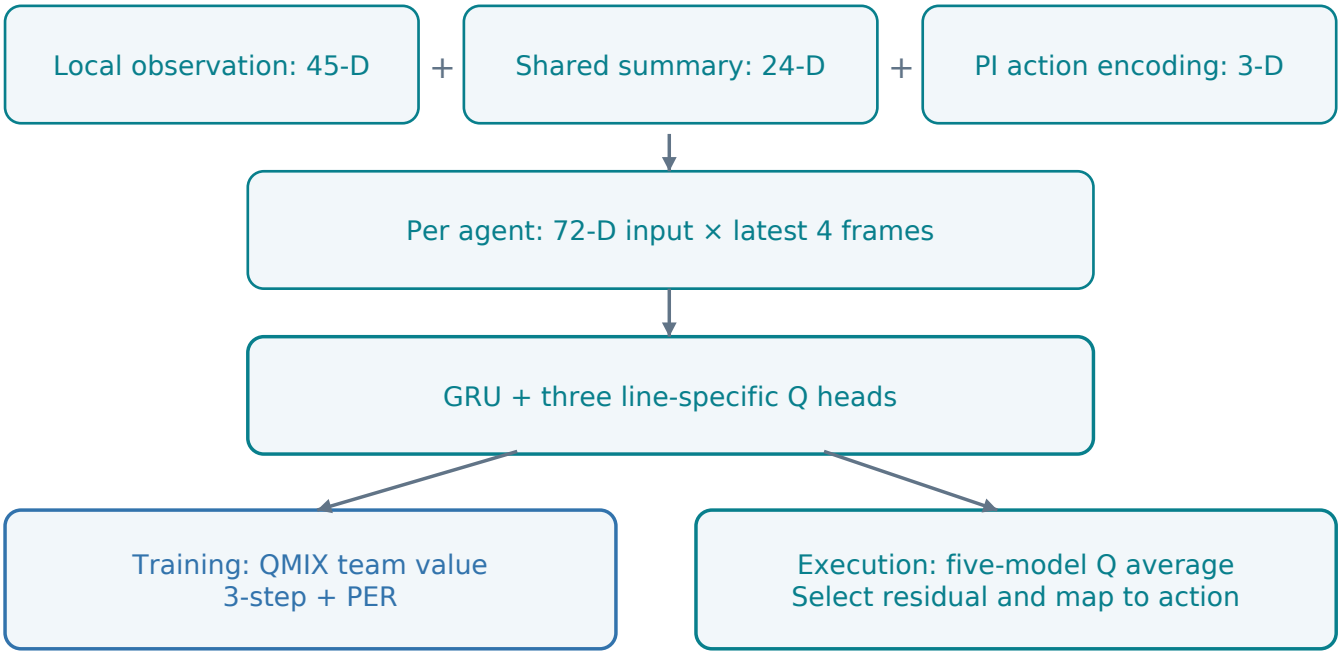


Figure 6. Historical M4 architecture. Dimensions, history and modules follow implementation. Training and execution branches have different roles.

Within a member, three agents share a GRU encoder but have line-specific heads. Five members train separately with different seeds. Execution still broadcasts global summaries. Decentralized action selection does not mean communication-free execution.

Version	Added to the preceding version	Problem addressed
M1	Global features + GRU	Incomplete information and unclear trends
M2	QMIX-style team-value mixing	Cross-line action coordination
M3	3-step return + PER	Delayed production feedback and learning efficiency
M4	Residual actions guided by current PI	Difficult exploration from scratch; reusable control knowledge
Final historical design	Longer training + five-member independent Q ensemble	Bias and instability from individual training runs

Chapter 6 / Information and memory

Global Dashboard + GRU: Understand Change

Mechanisms motivate the design; ablations must establish actual benefits.

An A supervisor sees low throughput and plans a cap increase. The shared dashboard shows A's buffer full, B undersupplied and assembly waiting for B. Extra A releases may not help. Coordination information enables decisions beyond local interests.

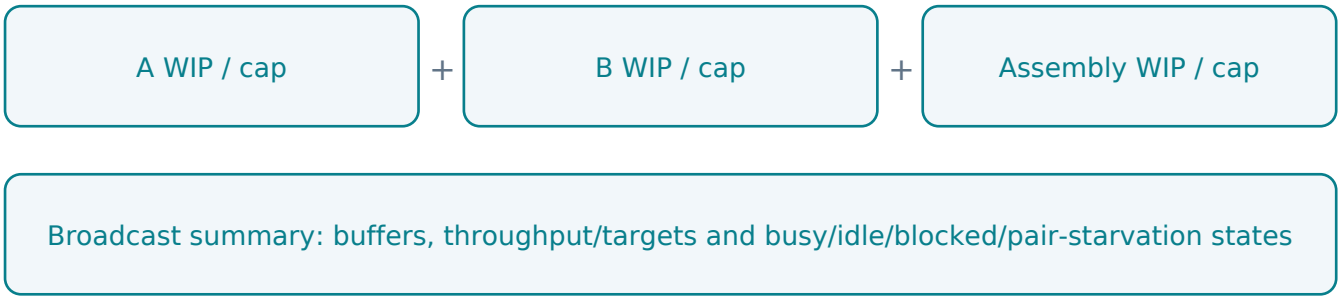
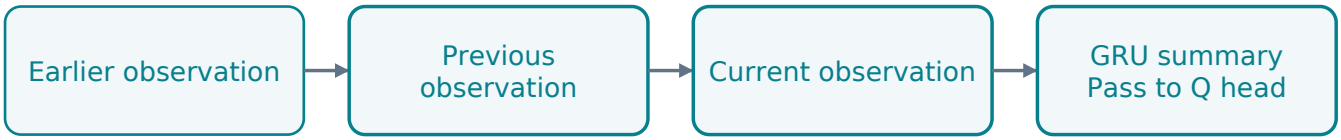


Figure 7. The 24-D global summary uses observable information, not future processing times or random-number access.

Why can the same snapshot require different actions?

Current throughput is below target in every case	Possible history	Control implication
Case A	Persistent shortage with no recovery	Supply may need sustained correction
Case B	Previously lower, now recovering	The previous adjustment may be taking effect
Case C	A single short-lived fluctuation	A large correction may overreact



The network re-encodes the latest 4 frames on every call. They span three 10-minute intervals, not unlimited episode-wide memory. GRU learns which changes to retain but is not guaranteed to identify every true cause.

Original MADQN already sees recent operating summaries. M1 adds more complete cross-line broadcasts and explicit sequence processing. Do not describe the original as completely blind.

Chapter 6 / Team learning

What QMIX, n-Step and PER Each Do

They change team-value representation, learning targets and replay sampling, respectively.

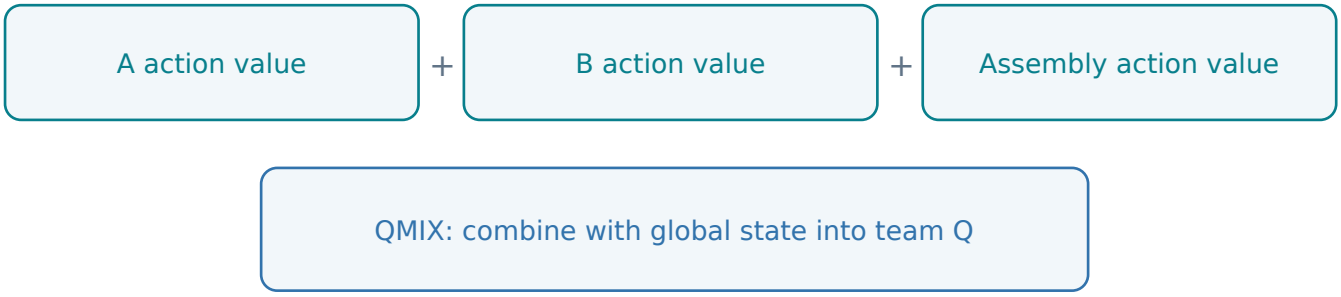


Figure 8. QMIX mixes line values into team value during training; it does not divide reward equally among agents.

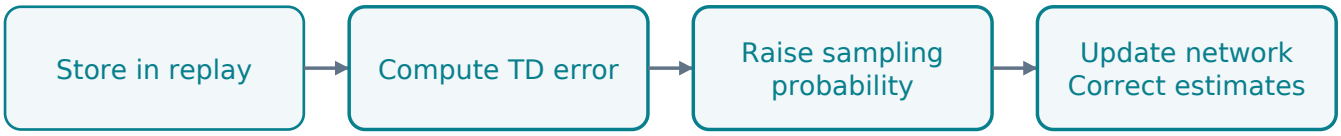
QMIX constrains the mixer to be monotonic, aligning local and team value directions so agents can select actions separately. It cannot express every possible joint-action relationship or produce a fully independent true-contribution reward for each supervisor.

3-step: bring delayed feedback into targets sooner

Target illustration: current reward + γ×next reward + γ²×following reward + γ³×successor-value estimate. γ=0.99. Termination removes subsequent bootstrapping.

Compared with one-step reward, three-step targets directly bring more actual feedback into an update. Each step is 10 minutes, but the algorithm does not look only 30 minutes ahead: the bootstrapped value represents a longer future.

PER: revisit experiences with inaccurate estimates



TD error is the gap between the current estimate and learning target. PER favors larger gaps and uses importance weights to mitigate sampling bias. Like a mistake notebook, it may also focus on noisy cases; it does not automatically identify the most important bottlenecks.

In the experiments, M2's mean reward worsens relative to M1, then M3 improves it. Evaluate designs with data rather than module count.

Chapter 6 / Experience guidance

PI Suggests; M4 Learns Corrections

The PI baseline uses fixed controller parameters. Historical files label it as a reconstruction. [S2, S3]

PI reads rolling throughput error. Its proportional term responds to current deviation and its integral term accumulates sustained deviation. The implementation also estimates workload allocation from processing means, adjusts target caps using buffer pressure, and uses filtering, deadbands and integral limits. It is not simply increase all caps whenever output is low.

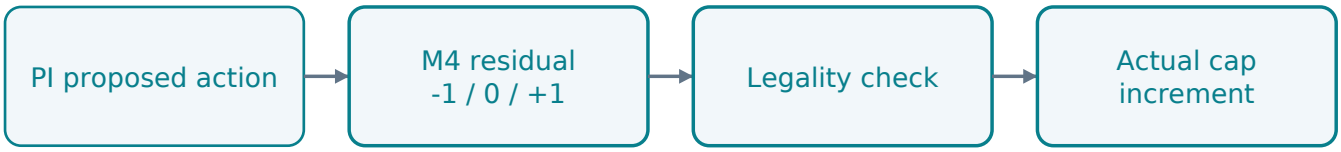


Figure 9. Residuals act on discrete actions. Zero follows PI; it is not an arbitrary real-valued cap addition.

PI proposal	Feasible residual	Possible executed action
Decrease cap	0 or +1	Decrease or hold
Hold	-1, 0 or +1	Decrease, hold, increase
Increase cap	-1 or 0	Hold or increase

If PI proposes increasing A, M4 can change it to hold but not directly to decrease, because the residual spans only adjacent actions. At cap=0 or cap=8, remove boundary-violating choices. This reduces exploration space but also limits corrections.

Why is this suitable for the environment?

PI already provides basic target/state-based cap control. M4 can focus on pairing imbalance, variability, delayed responses and coordination across regimes. Early training also uses PI guidance and a zero-residual preference.

PI guidance is not a formal safety proof and does not guarantee every correction beats PI. This historical M4 is a hybrid controller with a PI prior, not unaided pure MADQN.

Chapter 6 / Model ensemble

Five Sets of Scores, One Action

The historical final version averages Q values from five independent members, then selects per-line actions. [S4]

Different initialization and experiences produce different Q estimates even with the same algorithm. Ensembling aims to average away some individual bias. All members see the same state and PI proposal, then score each line's residual actions.

Illustrative model	Residual -1	Residual 0	Residual +1
Member A	0.60	0.58	0.10
Member B	0.45	0.59	0.20
Member C	0.48	0.57	0.18
Mean Q	0.51	0.58	0.16

Figure 10. Teaching example with three members. Average scores first, then select residual 0. Actual deployment uses five; these are not experimental Q values.

This is not majority voting: each member's score margin matters. Nor are output caps averaged and rounded. Execution produces one definite legal discrete action.

Independent Q versus joint Q

The selected independent Q ensemble averages local Q values per A/B/assembly unit and takes separate argmax choices. QMIX remains in each member's training. The earlier joint-Q experiment instead used mixers to evaluate joint actions. Their execution rules differ.

Same plateau-selected members: test results	reward×1000	NRMSE
Mean when 5 members execute separately	-38.6484	0.13233
Execution as five-member Q ensemble	-36.8359	0.12642

This more directly supports an ensemble benefit: the models are identical, execution differs. However, the rows summarize 60 member-seed episodes versus 12 ensemble episodes, not equal numbers of independent training experiments.

> Inference runs five members. Highly correlated errors may limit gains, and member Q scales affect averaging. Improvement is not universally guaranteed.

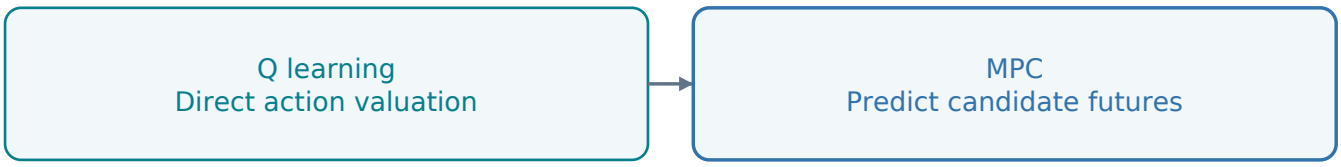
Chapter 7 / Baselines

Each Baseline Answers a Different Question

Baselines explain where improvement comes from, rather than merely increasing the comparison count.

Method	Supervisor analogy	Comparison purpose
Fixed CONWIP	Choose caps beforehand and hold them	Assess dynamic adjustment versus fixed settings
Current coordinated PI	Respond to error, accumulated deviation and material pressure	Compare with interpretable feedback control
Original MADQN	Three learners separately estimate local action values	Base reference for the improvement chain
Centralized DQN	One supervisor reads 135-D state and selects 27 joint actions	Compare centralized decisions and multi-agent structure
Learned-model MPC + PI	Predict, search future actions and constrain deviation using PI	Assess this predictive-control implementation

The simplest distinction between MPC and Q learning



Q networks compress long-term experience into action values and mainly evaluate them at execution. MPC predicts what an action will cause, searches action sequences, executes only the first action and replans. Their learned objects differ.

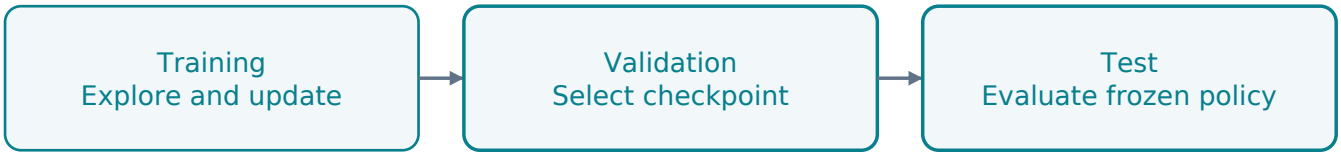
This MPC trains three feedforward predictors on 17280 transitions to predict observation increments and immediate reward. It uses a 4-step horizon and beam width 8, later adding a PI anchor, deviation penalties and a predicted-cost fallback. It does not implement the previously proposed GRU predictor.

Initial MPC performance was poor, and its controller was subsequently revised after inspecting test/holdout performance. Results are exploratory; neither set remains a fully unseen final test for MPC. PI safety region names an action constraint, not a formal safety guarantee. [S6]

Chapter 8 / Training

When Has a Learning Curve Reached a Plateau?

Training, validation and test rewards serve different purposes. [S4, S5]



Training reward varies with exploration and a changing policy. Validation disables exploration on fixed seeds, helping assess improvement. Test reports selected-model results. Repeated test-driven algorithm changes require a new final test set.

Actual validation curves: five members

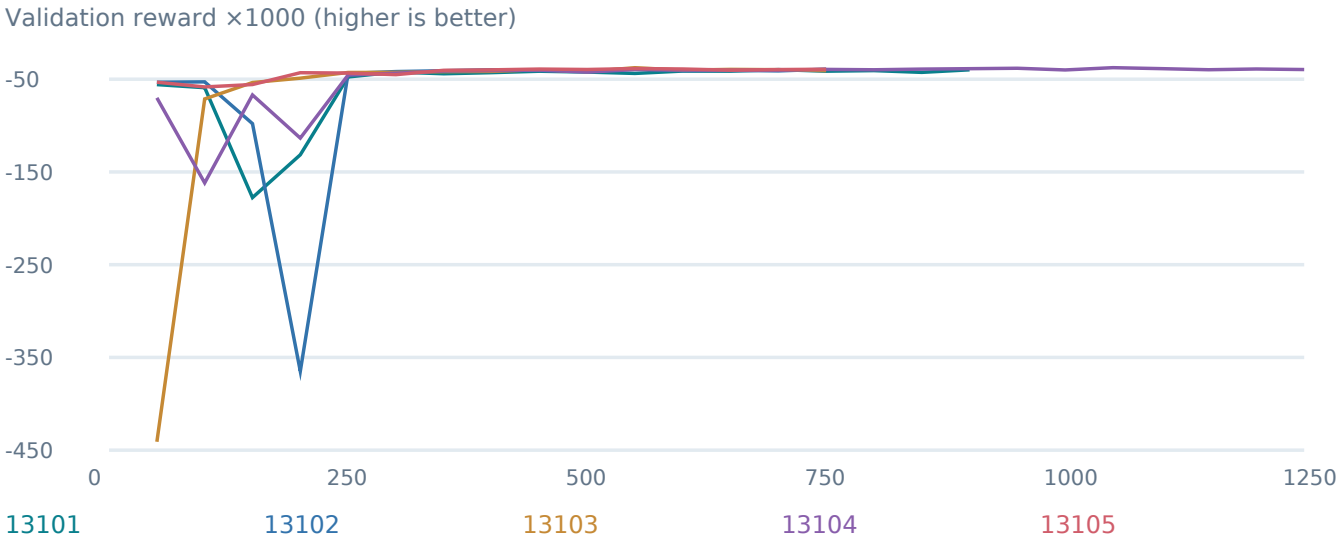


Figure 11. Unsmoothed validation reward against training episode. All observations remain, including early sharp drops.

Stopping: at least 300 episodes, validation every 50, and stop after 4 checks without sufficient improvement. Main threshold: reward×1000 gain of 0.05, with an NRMSE tie-break. Five members stopped at 900, 750, 750, 1250 and 750 episodes. Use their best validation checkpoints.

Convergence here means a practical validation-plateau stopping rule, not mathematical convergence, no possible future improvement or global optimality.

Chapter 8 / Results

Where Does Final Historical M4 Improve?

Existing test summary: higher reward and lower NRMSE are better. [S5, S6]

Method	reward×1000	NRMSE	Mean WIP	Change points
Fixed CONWIP	-116.570	0.31025	15.370	1.00
Current PI	-47.978	0.16769	14.452	129.58
Original MADQN	-65.659	0.19686	16.851	351.00
Central DQN	-62.991	0.17949	20.086	351.82
MPC + PI	-54.038	0.18339	14.626	157.42
Final historical M4 ensemble	-36.836	0.12642	14.692	231.92

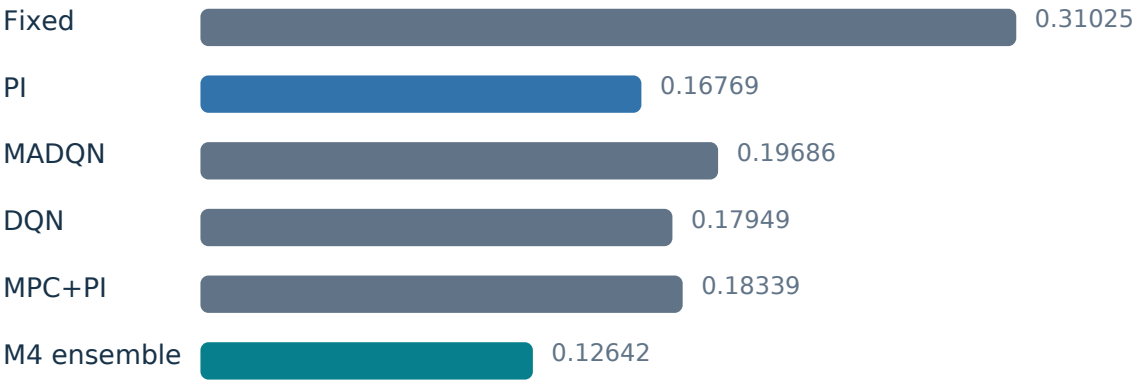


Figure 12. Actual mean NRMSE, bars starting at zero. Shorter is better. No error bars or significance claim.

Versus original MADQN, final historical M4 reduces NRMSE approximately 35.8%, aggregate cost 43.9%, mean WIP 12.8%, and change points 33.9%. These descriptive whole-design differences include longer training and ensemble benefits.

Versus PI, M4 reduces NRMSE approximately 24.6% and aggregate cost 23.2%, but WIP is approximately 1.7% higher and change points 79% higher. Its advantage is tracking, not uniformly lower operating metrics.

Original MADQN/DQN summarize 5 models × 12 seeds. Final ensembling uses one five-member combination on 12 seeds. Fixed policy's 1 change point follows existing statistics, not continuous dynamic adjustment. Rows are not strict equal-budget, equal-independent-run comparisons.

Chapter 8 / Evidence

Interpret Module Effects at the Right Level

Ablations examine module additions; final-system comparisons examine the entire package.

300-episode stage	reward×1000	NRMSE	Relative to previous version
M0 original MADQN	-65.659	0.197	Existing baseline
M1 global features + GRU	-50.098	0.153	Information/sequence version improves
M2 adds QMIX	-51.644	0.163	Tracking and reward do not improve further
M3 adds n-step + PER	-49.720	0.155	Improves over M2
M4 adds PI residuals	-43.066	0.145	Best within this version group

This table does not prove QMIX useless or every component useful. Components can interact. Removing QMIX from final M4 requires direct matched-budget, matched-seed comparison. M0 also differs in training implementation, so M0→M1 is not a clean single-variable ablation. [S3]

Final ensemble on two seed sets

Evaluation set	reward×1000 (mean ± SD)	NRMSE (mean ± SD)
test: 12 seeds	-36.8359 ± 1.7177	0.12642 ± 0.00677
holdout: 20 seeds	-36.3854 ± 2.1638	0.12451 ± 0.00858

Similar means support some stability across random processing samples within this scenario. SD measures episode variation, not confidence intervals. Samples are small and there is only one ensemble combination; newly trained ensembles need not perform identically.

A more defensible conclusion

Existing records show final historical M4 improves tracking and aggregate reward over the listed PI and learning baselines, and reduces inventory and changes versus original MADQN. NRMSE<0.10 remains unmet. Not every improvement has separately demonstrated positive benefit.

For confirmatory publication results, freeze algorithms/hyperparameters, use new seeds and regime schedules, match budgets and analyze paired outcomes. Current data illustrate motivation and observed effects.

Chapter 9 / Contributions

Place Contributions Where the Evidence Supports Them

A combined design and empirical validation for a specific production-control problem.

GRU, QMIX, n-step, PER, residual control and ensembles are established techniques. The contribution is aligning them with distinct A+B assembly challenges and organizing a working controller under a shared production objective. This does not establish literature-level priority.

Environment challenge	Corresponding design	Defensible value
Upstream/downstream pairing and finite buffers	Global summaries + team-value training	Help each line learn from plant-wide information
Processing delays and timing variability	GRU + multi-step returns	Use recent dynamics and more distant feedback
Inefficient exploration from scratch	PI action residuals and training guidance	Learn corrections on top of feedback control
Single-model training differences	Five-member independent Q averaging	This member combination executes better than the individual mean

Research summary

This project studies dynamic CONWIP-cap control in stochastic paired assembly. A three-loop simulation with changing operating regimes captures finite buffers, material synchronization and delayed feedback. The historical MADQN-based design integrates global features, short-history GRU, QMIX team training, multi-step prioritized replay and PI residual guidance. Averaging local Q values across five independently trained members improves tracking and aggregate reward in existing scenarios, with explicit discussion of inventory, adjustments and evaluation fairness.

Which benefits remain unverified?

Real-factory deployment, breakdown robustness, transfer to new topologies, arbitrary target schedules and large-scale multi-line expansion remain untested. Cap actions do not replace maintenance, equipment configuration or order scheduling.

For mechanisms, say aims to mitigate or may help. For results, say observed in this experiment. Plausible design and experimental proof are different levels of evidence.

Appendix / Review and evidence

Review the Project with These Six Questions

The guide describes the implemented system and its recorded experiments.

Question	Answer
Are cap and WIP identical?	No. Cap is an allowance; WIP is actual inventory. WIP may temporarily exceed a reduced cap.
Why not set every cap to 8?	More inventory may not remove a bottleneck, and raises inventory cost and overproduction risk.
Does M4 require PI?	This historical version does: PI supplies inputs and an action anchor. The latest PI-off version does not.
Does independent Q ensembling remove QMIX?	No. Members train with QMIX; execution averages Q per line.
Does convergence imply optimality?	No. It means a practical validation-plateau stopping rule here.
Does M4 outperform PI on everything?	Tracking and reward improve; PI has lower inventory and fewer change points.

Local source index

[S1] work/conwip_four_strategy_v3_madqn/conwip/pair_v2.py: topology, parameters, actions, observations and reward.
[S2] baseline_algorithms.py in the same directory: current PI, original MADQN and central DQN.
[S3] work/conwip_four_strategy_v3_madqn/run_madqn_ablation.py; outputs/madqn_ablation/MADQN_Ablation_Report.md: M1-M4 definitions and early results.
[S4] work/train_m4_until_converged.py; work/evaluate_m4_converged_final.py: stopping, member selection and Q ensembling.
[S5] outputs/madqn_m4_final_converged/: training_manifest.json, comparison_final.json, validation.csv and comparison tables.
[S6] work/train_mpc_controller.py; outputs/mpc_model_predictive/comparison_mpc.json: MPC implementation and exploratory results.

Paths are relative to the project root. The PI baseline is reconstructed from the available controller parameters. Experiment values are reported without modification.

You can now explain why a busy A line may not increase finished output, why current error alone is insufficient, why M4 uses shared information and memory, and why better results do not attribute all gains to one module.